

From raw ALM defect files to AI-enriched PDF reports in under 60 seconds — combined with IBM ICA-powered requirements intelligence in one secured workspace.

At a Glance

5	<60s	0	2
AI task modes for requirements	Full RCA cycle per defect file	External DBs or dependencies	IBM AI engines (watsonx + ICA)

Before vs. After

BEFORE - x Manual CSV triage — hours per sprint - x Inconsistent RCA across analysts - x No requirements AI tooling - x Separate tools, no shared login - x Static, templated reports	>>	AFTER - v Sub-60-second automated analysis + PDF - v Consistent, AI-enriched RCA every time - v 5-task IBM ICA Agent requirements review - v Unified login, one interface, one server - v AI executive summary in every report
--	----	---

What Makes It Stand Out

- **Graceful AI degradation by design.** The system has two fully working modes. With IBM watsonx.ai credentials, it calls Granite-3-8B-Instruct for AI-generated 5-Whys, preventive measures, root cause suggestions, and an executive summary, and Slate-125M for semantic defect clustering. Without credentials, it falls back silently to rule-based keyword grouping and static templates — no errors, no crashes, no user action required. This was an intentional engineering decision: the tool is always production-ready regardless of which AI services are available.
- **End-to-end IBM alignment — no third-party AI.** Every intelligent call routes exclusively through IBM infrastructure: IBM watsonx.ai handles defect intelligence (text generation + semantic embeddings) and IBM ICA Agent handles requirements analysis. No OpenAI, no Anthropic, no HuggingFace. Fully compliant with IBM's AI governance, data residency, and acceptable-use policies.
- **Semantic defect clustering without a vector database.** The Hub uses IBM Slate-125M embeddings and cosine similarity computed in pure Python (no NumPy, no FAISS, no Pinecone) to group semantically related defects at runtime. The clustering result directly shapes which defects are presented as RCA candidates and which 5-Whys narratives are generated.

The AI Intelligence Pipeline

Every defect analysis run passes through a five-stage pipeline. Stages 3 and 4 adapt dynamically based on which IBM AI services are configured:

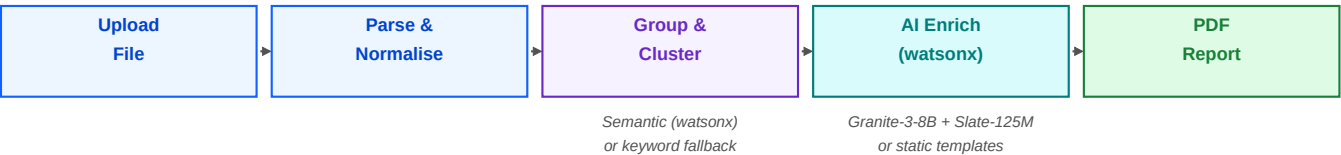


Figure 1 — The pipeline is identical in both modes; only the intelligence depth changes.

Stage	What Happens	Technology	Mode
1 Upload	File received, UUID session ID assigned, stored with timestamped name	Flask · Werkzeug	Always
2 Parse	Multi-encoding CSV detection (UTF-16/8/Latin-1), flexible column mapping, severity/status/module/trend metrics computed	pandas	Always
3 Group	Defects grouped by semantic similarity (cosine on Slate-125M embeddings) or keyword fallback. Groups become RCA clusters.	watsonx.ai or keywords	Adaptive
4 AI Enrich	Per-cluster 5-Whys (Granite-3-8B), preventive measures, per-defect root cause suggestion (top 20), AI executive summary for PDF header	IBM Granite 3-8B-Instruct	Adaptive
5 Report	ReportLab PDF rendered with charts, RCA table, measures, and AI summary (or static summary if no watsonx credentials)	ReportLab 4.2	Always

Key Engineering Decisions

No vector DB required Cosine similarity over Slate-125M embeddings computed in pure Python stdlib (math.sqrt, zip). Eliminates NumPy, FAISS, and Pinecone as dependencies — the Hub installs with a single pip command.	Session-based isolation Each analysis uses a UUID-prefixed filename. No shared state between concurrent users. No database writes. Results are JSON files on disk — trivially auditable and portable.
API keys never reach the browser GET /api/settings returns ica_api_key_set: bool and watsonx_api_key_set: bool — never the actual key values. Secrets live only in config.json on the server.	Retry logic mirrors production ICA patterns ICA Agent calls retry up to 2x with a 3-second gap on HTTP 502/503/504. Timeout is 120s per attempt. Errors are converted to clean user-facing messages — raw gateway HTML never reaches the UI.

Where It Sits in the SDLC

The Hub intervenes at two critical — and typically underserved — points in the software delivery lifecycle, shifting quality effort left:

SDLC Phase	What the Hub Does	Impact
Requirements (Pre-dev)	IBM ICA Agent reviews specifications for ambiguity, edge cases, test gaps, and cross-consistency before a single line of code is written	Defects caught at requirements cost ~1x to fix vs. ~100x in production
Post-sprint (Defect triage)	ALM export uploaded, analysed, and AI-enriched report generated in <60s with clustered RCA candidates and preventive measures	Hours of manual triage eliminated per sprint; consistent outputs across teams
Continuous (All sprints)	PDF reports accumulate as a historical quality record; monthly trend data surfaces recurring failure patterns across releases	Data-driven retrospectives replace anecdotal quality conversations

The ICA Agent Router Pattern

A key design insight in the Requirements Analyzer is that a single IBM ICA Agent endpoint can behave as five different specialists — depending entirely on the system prompt prepended to the user's input. This is the **router pattern**:

Task Key	System Prompt Persona	User Input	Output
translate	Requirements Translator — preserves all technical details	Requirements in any language	Professional English requirements
analyze	Requirements Analyst — clarity, completeness, ambiguities, improvements	Requirements document	Structured quality report
edge_cases	QA Expert — boundary conditions, negative scenarios, systematic coverage	Requirements or feature description	Edge case catalogue
review_tests	Test Case Reviewer — gaps, duplicates, edge cases, inconsistencies	Requirements + test cases	Test coverage gap report
validate	Validation Expert — uncovered requirements, orphan tests, contradictions	Requirements + test cases	Consistency validation report

Why this matters: One agent endpoint. Five specialised intelligences. No fine-tuning, no custom models, no additional infrastructure. The entire routing logic is 53 lines of Python in *ica_agent.py*.

Security Model & Deployment Footprint

Authentication pbkdf2:sha256 hashed passwords (Werkzeug). Flask server-side sessions. Role-based access (admin / user). Default admin auto-seeded on first run. All routes protected by @login_required decorator.	Secret Management API keys stored only in config.json on the server filesystem. GET /api/settings returns boolean presence flags — never key values. User passwords stored as hashes — never plaintext anywhere.	Deployment Footprint Single Python process. No Docker required. No cloud account needed for rule-based mode. Runs on any machine with Python 3.8+. Start command: python app.py
--	--	---

Future Scope

IBM watsonx Orchestrate Integration	The Hub currently calls IBM AI services via direct API. The next evolution is to expose it as a watsonx Orchestrate skill — allowing QA engineers to trigger defect analysis and requirements reviews directly from a conversational AI assistant, embedded within their existing IBM tools workflow without opening a browser at all.
Cross-Release Trend Intelligence	Each analysis already produces a timestamped JSON output. By connecting these outputs to a lightweight time-series store (even a simple SQLite DB), the Hub could surface defect velocity trends, recurring root cause patterns across multiple releases, and predictive quality risk scores — turning retrospective reports into forward-looking quality forecasts.

Built in IBM. Runs on IBM AI. Designed for IBM delivery teams. The Quality Intelligence Hub is not a proof of concept — it is a production-ready tool actively used to analyse real ALM defect data.

Internal IBM tooling — not for external distribution.